

---

## `std::execution::task` Shipping Gate

---

Document Number: D4007R1  
Date: 2026-03-11  
Audience: LEWG  
Reply-to: Vinnie Falco [vinnie.falco@gmail.com](mailto:vinnie.falco@gmail.com)  
Mungo Gill [mungo.gill@me.com](mailto:mungo.gill@me.com)

### Table of Contents

---

Abstract

Revision History

R1: March 2026 (pre-Croydon mailing)

R0: February 2026 (pre-Croydon mailing)

1. Disclosure

2. Fixed After Ship

3. Ships Broken

4. Straw Polls

5. Proposed Wording

References

WG21 Papers

Other

### Abstract

---

`std::execution::task` (P3552R3<sup>[1]</sup>) has open defects identified by national ballot comments, LWG issues, and published papers. This paper lists criteria that must be resolved to ship `task` in C++26. If all are resolved, ship. If any remain, apply the proposed wording in Section 5 to remove `task` from the working draft.

---

## Revision History

---

### R1: March 2026 (pre-Croydon mailing)

- Complete rewrite as a criteria paper.

### R0: February 2026 (pre-Croydon mailing)

- Original analysis of structural gaps. See [P4007R0](#)<sup>[2]</sup>.
- 

## 1. Disclosure

---

The authors developed [P4003R0](#)<sup>[3]</sup> (“Coroutines for I/O”) and [P4007R0](#)<sup>[2]</sup> (“Senders and Coroutines”). A coroutine-only design cannot express compile-time work graphs, does not support heterogeneous dispatch, and assumes a cooperative runtime. Those are real costs. The defects documented below hold regardless of whether P4003 is the right design or any design at all.

---

## 2. Fixed After Ship

---

`task`’s `promise_type` is a class template instantiated in user code. Its `operator new`, allocator selection, stop token storage, environment forwarding, and destruction ordering can change between standard revisions without binary incompatibility. These issues are fixable post-ship: [Unusual Allocator Customisation](#)<sup>[4]</sup>, [Flexible Allocator Position](#)<sup>[4]</sup>, [Shadowing The Environment Allocator](#)<sup>[4]</sup>, [Stop Source Always Created](#)<sup>[4]</sup>, [Stop Token Default Constructible](#)<sup>[4]</sup>, [Task Not Actually Lazily Started](#)<sup>[4]</sup>, [Frame Destroyed Too Late](#)<sup>[4]</sup>, [No Default Arguments](#)<sup>[4]</sup>, `unhandled_stopped` `Not noexcept`<sup>[4]</sup>, [Environment Design Inefficient](#)<sup>[4]</sup>, [Non-Sender Awaitables Unsupported](#)<sup>[4]</sup>, [Future Language Feature Could Avoid `co\_yield`](#)<sup>[4]</sup>, [No TLS Capture/Restore Hook](#)<sup>[4]</sup>, `return_value / return_void` [Have No Specification](#)<sup>[4]</sup>, `co_return { args... }` [Unsupported](#)<sup>[4]</sup>, `change_coroutine_scheduler` [Requires Assignable Scheduler](#)<sup>[4]</sup>, [Sender-Unaware Coroutines Cannot `co\_await a task`](#)<sup>[4]</sup>, [Missing Rvalue Qualification](#)<sup>[4]</sup>, [Parameter Lifetime Is Surprising](#)<sup>[5]</sup>, [No Protection Against Dangling References](#)<sup>[5]</sup>, `co_yield with_error` [Is Clunky](#)<sup>[5]</sup>, `co_await schedule(sch)` [Is an Expensive No-Op](#)<sup>[5]</sup>, [Coroutine Cancellation Is Ad-Hoc](#)<sup>[5]</sup>.

`affine_on` semantics, rescheduling behavior, and algorithm customisation are specification-level concerns. Tightening requirements or adding default implementations does not change any published interface. These issues are fixable post-ship: `affine_on` Default Implementation Lacks Specification<sup>[4]</sup>, `affine_on` Semantics Not Clear<sup>[4]</sup>, `affine_on` Shape May Not Be Correct<sup>[4]</sup>, `affine_on` Shouldn't Forward Stop Requests<sup>[4]</sup>, `affine_on` Customisation For Other Senders<sup>[4]</sup>, Starting a `task` Reschedules Unconditionally<sup>[4]</sup>, Resuming After a `task` Reschedules Unnecessarily<sup>[4]</sup>, `bulk` vs. `task_scheduler`<sup>[4]</sup>, No Completion Scheduler<sup>[4]</sup>, `with_awaitable_senders` Unused<sup>[4]</sup>.

### 3. Ships Broken

The criteria in this section are items where shipping forecloses the fix.

| Criterion                | References                                                                                                                                                                                         | Fixed |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Allocator<br>Timing      | <code>P4007R0</code> <sup>[2]</sup> S5, <code>D3980R0</code> <sup>[6]</sup> , <code>P3796R1</code> <sup>[4]</sup> , <code>LWG 4356</code> <sup>[7]</sup> , <code>US 254-385</code> <sup>[8]</sup>  | no    |
| Allocator<br>Propagation | <code>P4007R0</code> <sup>[2]</sup> S5, <code>D3980R0</code> <sup>[6]</sup> , <code>P3796R1</code> <sup>[4]</sup>                                                                                  | no    |
| Complicated<br>Success   | <code>P4007R0</code> <sup>[2]</sup> S3, <code>P2430R0</code> <sup>[9]</sup>                                                                                                                        | no    |
| Exception<br>Hygiene     | <code>P4007R0</code> <sup>[2]</sup> S3.7                                                                                                                                                           | no    |
| Error Return             | <code>P4007R0</code> <sup>[2]</sup> S4, <code>P3950R0</code> <sup>[10]</sup> , <code>P3801R0</code> <sup>[5]</sup> , <code>P1713R0</code> <sup>[11]</sup>                                          | no    |
| Symmetric<br>Transfer    | <code>P2583R3</code> <sup>[12]</sup> , <code>US 246-373</code> <sup>[13]</sup> , <code>LWG 4348</code> <sup>[14]</sup> , <code>P3801R0</code> <sup>[5]</sup> , <code>P3796R1</code> <sup>[4]</sup> | no    |

- **Allocator Timing.** The user can specify either a type `Allocator` or pointer to `pmr::memory_resource` at the launch site and this allocator is injected into the environment and used to allocate the coroutine frame of the launched task; if the launch function implementation creates any trampolines, those coroutine frames are also allocated using the user's specified allocator.
- **Allocator Propagation.** The allocator used in the launch function is invisibly propagated to all child tasks.
- **Complicated Success.** I/O outcomes ( `error_code` , value pair) map cleanly to the sender's three-channel design, without data loss, and without foreclosing a subset of downstream sender algorithms.

- **Exception Hygiene.** No exceptions thrown for transient errors like `ECONNRESET` or `ssl::stream_truncated`.
  - **Error Return.** Users write `co_return` to propagate error codes to the caller, not `co_yield`.
  - **Symmetric Transfer.** `sender-awaitable::await_suspend` returns `void`, so `co_awaiting` a synchronously completing sender grows the stack by one frame per completion with no upper bound.
- 

## 4. Straw Polls

---

1. “The committee is comfortable shipping `std::execution::task` in C++26 with the defects listed in Section 3 unresolved.”
  2. “Apply the proposed wording in Section 5.”
- 

## 5. Proposed Wording

---

TBD

---

## References

---

### WG21 Papers

1. **P3552R3** - “Add a Coroutine Task Type” (Dietmar Kühl, Maikel Nadolski, 2025). <https://wg21.link/p3552r3>
2. **P4007R0** - “Senders and Coroutines” (Vinnie Falco, Mungo Gill, 2026). <https://wg21.link/p4007r0>
3. **P4003R0** - “Coroutines for I/O” (Vinnie Falco, Steve Gerbino, Mungo Gill, 2026). <https://wg21.link/p4003r0>
4. **P3796R1** - “Coroutine Task Issues” (Dietmar Kühl, 2025). <https://wg21.link/p3796r1>
5. **P3801R0** - “Concerns about the design of `std::execution::task`” (Jonathan Müller, 2025). <https://wg21.link/p3801r0>
6. **D3980R0** - “Task’s Allocator Use” (Dietmar Kühl, 2026). <https://isocpp.org/files/papers/D3980R0.html>
7. **LWG 4356** - “`connect()` should use `get_allocator(get_env(rcvr))`”. <https://cplusplus.github.io/LWG/issue4356>
8. **US 254-385** - C++26 NB ballot comment. <https://github.com/cplusplus/nbballot/issues/960>
9. **P2430R0** - “Partial success scenarios with P2300” (Chris Kohlhoff, 2021). <https://wg21.link/p2430r0>

10. **P3950R0** - “return\_value & return\_void Are Not Mutually Exclusive” (Robert Leahy, 2025).  
<https://wg21.link/p3950r0>
11. **P1713R0** - “Allowing both co\_return; and co\_return value; in the same coroutine” (Lewis Baker, 2019).  
<https://wg21.link/p1713r0>
12. **P2583R3** - “Symmetric Transfer and Sender Composition” (Mungo Gill, Vinnie Falco, 2026).  
<https://wg21.link/p2583r3>
13. **US 246-373** - C++26 NB ballot comment. <https://github.com/cplusplus/nbballot/issues/948>
14. **LWG 4348** - “task doesn’t support symmetric transfer”. <https://cplusplus.github.io/LWG/issue4348>

## Other

15. **P2300R10** - “std::execution” (Michał Dominiak et al., 2024). <https://wg21.link/p2300r10>
16. **C++ Working Draft** - (Richard Smith, ed.). <https://eel.is/c++draft/>